# FALCON 9 | LANDING PREDICTION

IBM APPLIED DATA SCIENCE CAPSTONE • PROJECT PRESENTATION

- End-to-end study: API extraction • scraping • cleaning • SQL • visualization • classification
- Objective: predict first-stage landing outcome from launch characteristics
- Prepared from six uploaded IBM lab notebooks; unexecuted results are transparently identified

IMPORTANT: Final quantitative findings require executing the uploaded notebooks.

# EXECUTIVE SUMMARY

What the project investigates

- Business problem: booster recovery is central to reusable-launch economics; prediction supports understanding of launch conditions.
- Data sources: SpaceX launch API and Falcon 9 Wikipedia launch tables; IBM-provided curated datasets used in later labs.
- Methods: Pandas preprocessing, SQL, Seaborn EDA, Folium geospatial analysis, Plotly Dash exploration, four classification algorithms.
- Deliverable status: code/methods verified from notebooks; empirical model scores and charts are unavailable because supplied notebooks lack executed outputs.

SOURCE: Uploaded IBM course notebook(s) | Results not executed in supplied files

# INTRODUCTION | DECISION CONTEXT

From launches to a binary classification target

- Question: given flight and mission attributes, will the first stage land successfully?
- Target: Class = 1 for successful recovery and 0 otherwise, as defined in the IBM project workflow.
- Candidate predictors: FlightNumber, PayloadMass, Orbit, LaunchSite, Flights, GridFins, Reused, Legs, LandingPad, Block, ReusedCount and Serial.
- The analysis is retrospective; correlations do not establish that any feature causes a successful landing.

SOURCE: Uploaded IBM course notebook(s) | Results not executed in supplied files

# PROJECT WORKFLOW

Six notebooks provided; dashboard source not provided

- 01 Collect API launch records and enrich rocket / payload / launchpad / core identifiers.
- 02 Parse launch tables from Wikipedia and assemble tabular records.
- 03 Clean, derive target labels and encode categorical variables.
- 04 Analyze SQL summaries and Seaborn charts; explore launch locations using Folium.
- 05 Fit and compare logistic regression, SVM, decision tree and k-nearest neighbors.

SOURCE: Uploaded IBM course notebook(s) | Results not executed in supplied files

# DATA COLLECTION | SPACEX API

Notebook: jupyter-labs-spacex-data-collection-api-v2

- Use requests to obtain launch records and normalize nested JSON into a dataframe.
- Helper functions retrieve related metadata: booster details, launchpad, payload and core information.
- Retain launch date, flight number, payload mass, orbit, landing outcome and reuse-related variables.
- Check HTTP status and completeness before exporting; one saved response is HTML

GET launches → resolve nested IDs → build DataFrame → filter Falcon 9 → CSV

SOURCE: Uploaded IBM course notebook(s) | Results not executed in supplied files

# DATA COLLECTION | WEB SCRAPING

Notebook: jupyter-labs-webscraping

- Request the Wikipedia list of Falcon 9 / Falcon Heavy launches.
- Use BeautifulSoup to identify launch tables and parse header / row cells.
- Extract launch identifiers, booster version, payload, payload mass, orbit, customer and mission outcome.
- Standardize strings, units and missing values before constructing a Pandas dataframe.

requests.get(url) → BeautifulSoup(html) → table rows → launch_dict → DataFrame

SOURCE: Uploaded IBM course notebook(s) | Results not executed in supplied files

# DATA WRANGLING | CLEANING

API and EDA notebooks

- Filter to the Falcon 9 first-stage analysis population.
- Replace missing PayloadMass with the column mean as specified by the IBM collection lab.
- Create binary Class labels from landing outcomes; review ambiguous landing outcomes rather than guessing.
- Select modeling predictors, one-hot encode Orbit / LaunchSite / other categorical features and cast numeric columns to float64

```
features = pd.get_dummies(features, columns=['Orbit','LaunchSite', ...])
```

SOURCE: Uploaded IBM course notebook(s) | Results not executed in supplied files

# EDA METHODOLOGY | WHY THESE CHARTS?

Notebook: jupyter-labs-eda-dataviz-v2

- Scatter: FlightNumber × PayloadMass colored by Class → experience and payload patterns.
- Categorical scatter: FlightNumber × LaunchSite and PayloadMass × LaunchSite → site-specific distributions.
- Bar: landing success fraction by Orbit → compare outcomes across mission types.
- Line: yearly average Class → visualize how historical landing outcomes evolve.

SOURCE: Uploaded IBM course notebook(s) | Results not executed in supplied files

# EDA | FLIGHT EXPERIENCE & PAYLOAD

Required scatter-plot result interpretation

- The lab describes a qualitative trend toward more successful landings at later flight numbers.
- The lab also proposes that payload mass may relate to recovery outcome; assess visually with class-colored markers.
- Recommended axes: FlightNumber (x), PayloadMass in kg (y), Class (color).
- RESULT STATUS: chart absent from uploaded notebook outputs; no numeric effect size can be verified

```
sns.catplot(x="FlightNumber", y="PayloadMass", hue="Class", data=df)
```

SOURCE: Uploaded IBM course notebook(s) | Results not executed in supplied files

# EDA | LAUNCH SITE & ORBIT

Required site and orbit comparisons

- FlightNumber × LaunchSite identifies launch chronology and outcome at each facility.
- PayloadMass × LaunchSite reveals which sites handled different payload ranges.
- The lab text notes no >10,000 kg payload launches at VAFB-SLC in its example dataset; verify on rerun.
- Success by Orbit should be computed as mean(Class) per orbit, not inferred from counts alone

```
df.groupby('Orbit')['Class'].mean().sort_values()
```

SOURCE: Uploaded IBM course notebook(s) | Results not executed in supplied files

# EDA | YEARLY LANDING TREND

Trend interpretation and limitations

- Convert Date to year and calculate mean Class for each calendar year.
- Plot annual recovery success fraction to inspect operational learning over time.
- Interpret early/late years carefully: sample sizes and mission mix may differ.
- RESULT STATUS: notebook contains chart instructions but no saved output or annual rates.

```
df.groupby('Year')['Class'].mean().plot(kind='line')
```

SOURCE: Uploaded IBM course notebook(s) | Results not executed in supplied files

# EDA WITH SQL | QUERY DESIGN

Notebook: jupyter-labs-eda-sql-coursera_sqllite

- SQL exploration answers structured questions about launch sites, payloads and mission outcomes.
- Core operations: SELECT, DISTINCT, WHERE, GROUP BY, ORDER BY, COUNT, SUM and AVG.
- Use launch-site aggregation to measure launch frequency; calculate outcome ratios only with a valid success indicator.
- RESULT STATUS: supplied SQL notebook has no executed query outputs; do not claim specific counts

```
SELECT Launch_Site, COUNT(*) AS launches FROM SPACEXTBL GROUP BY Launch_Site;
```

SOURCE: Uploaded IBM course notebook(s) | Results not executed in supplied files

# SQL | SITE AND PAYLOAD INVESTIGATION

Examples aligned with required rubric

- Which launch sites appear in the dataset? Use SELECT DISTINCT Launch_Site.
- How many launches occurred at each site? Use GROUP BY Launch_Site and COUNT(*).
- Which payloads were heaviest? Sort PAYLOAD_MASS__KG_ descending; inspect mission context.
- Which dates and booster outcomes matter? Filter dates and Landing_Outcome values explicitly.

```
SELECT Launch_Site, MAX(PAYLOAD_MASS__KG_) FROM SPACEXTBL GROUP BY Launch_Site;
```

SOURCE: Uploaded IBM course notebook(s) | Results not executed in supplied files

# SQL | RANKINGS AND TIME ANALYSIS

Queries should return tables and conclusions

- Rank sites by launch volume and compare their mission mix.
- Filter mission years to inspect changes in landing outcomes across time.
- Compare total successful outcomes to all outcomes using explicit success criteria.
- Do not equate mission success with first-stage landing success: these are different labels.

```
SELECT Landing_Outcome, COUNT(*) FROM SPACEXTBL GROUP BY Landing_Outcome;
```

SOURCE: Uploaded IBM course notebook(s) | Results not executed in supplied files

# FOLIUM | GEOSPATIAL METHOD

Notebook: lab-jupyter-launch-site-location-v2

- Read launch-site coordinates: Launch Site, Lat, Long and class.
- Place a marker / circle at each unique launch site with an informative popup.
- Use MarkerCluster to show individual successful and unsuccessful launches by location.
- Calculate distances from sites to geographic features; interpret proximity as descriptive, not causal.

folium.Map(...) → folium.Marker(...) → MarkerCluster(...) → distance()

SOURCE: Uploaded IBM course notebook(s) | Results not executed in supplied files

# FOLIUM | PROXIMITY ANALYSIS

Required map and distance evidence

- The notebook defines tasks for site markers, launch-record markers and distances to nearby features.
- Use a coordinate-based distance formula and annotate the measured start/end points.
- Compare proximity to coastlines, roads or railways only when coordinates are available and verified.
- RESULT STATUS: no saved map render, launch markers or measured distances in the uploaded notebook.

SOURCE: Uploaded IBM course notebook(s) | Results not executed in supplied files

# PLOTLY DASH | INTERACTIVE ANALYTICS

Dashboard source was not included in the six uploaded files

- Intended control: choose launch site (all sites or one site) and filter payload-mass range.
- Success pie chart: show share of successful versus unsuccessful landings for the selected site.
- Payload scatter: plot PayloadMass against Class, colored by booster version or other relevant group.
- RESULT STATUS: actual dashboard screenshots and Python app cannot be verified without the Dash lab/app.py.

SOURCE: Uploaded IBM course notebook(s) | Results not executed in supplied files

# PREDICTIVE ANALYSIS | MODELING DESIGN

Notebook: SpaceX-Machine-Learning-Prediction-Part-5

- Outcome: Class; numeric predictors standardized with StandardScaler.
- Split: 80% train and 20% held-out test; random_state=2 per lab.
- Tune models with GridSearchCV using 10-fold cross-validation on training data.
- Candidate algorithms: Logistic Regression, SVM, Decision Tree and KNN.
- Compare cross-validation scores, then test-set accuracy and confusion matrices.

SOURCE: Uploaded IBM course notebook(s) | Results not executed in supplied files

# PREDICTIVE ANALYSIS | MODEL COMPARISON

Four classifiers in uploaded notebook

- Logistic Regression: linear decision boundary; tune regularization strength C.
- SVM: margin-based classifier; tune kernel, C and gamma.
- Decision Tree: nonlinear decision rules; tune depth and splitting constraints.
- KNN: local neighborhood voting; tune n_neighbors and distance settings.
- RESULT STATUS: no fit outputs or scores saved; best model cannot be established.

SOURCE: Uploaded IBM course notebook(s) | Results not executed in supplied files

# CONFUSION MATRIX | HOW TO INTERPRET

Required evaluation artifact

- True Positive: landing predicted and observed. True Negative: failure predicted and observed.
- False Positive: predicted landing but booster did not land — particularly important for risk assessment.
- False Negative: predicted failure but booster landed.
- Report TP, TN, FP, FN and test accuracy; precision and recall can add operational context.

```
confusion_matrix(Y_test, model.predict(X_test))
```

SOURCE: Uploaded IBM course notebook(s) | Results not executed in supplied files

# CONCLUSION | WHAT THE LABS SUPPORT

Evidence-based synthesis without fabricated statistics

- The workflow demonstrates how heterogeneous launch data can be collected, cleaned, queried, visualized and modeled.
- Flight chronology, payload mass, orbit and launch site are useful candidate variables to investigate.
- Four classification families provide a baseline for comparison using consistent holdout evaluation.
- No defensible claim about the winning model, numerical accuracy or site rankings is possible until outputs are executed and captured.

SOURCE: Uploaded IBM course notebook(s) | Results not executed in supplied files

# INNOVATIVE INSIGHTS | NEXT QUESTIONS

Follow-up analysis that adds value

- Separate the effect of experience (FlightNumber) from changing payload/orbit mix by stratified comparisons.

- Inspect false positives by launch site and orbit to find potentially high-risk operating regimes.

- Evaluate calibration: are predicted landing probabilities reliable, not just classification labels?

- Compare model performance over chronological splits to test whether historical accuracy transfers to later launches.

SOURCE: Uploaded IBM course notebook(s) | Results not executed in supplied files

# SUBMISSION | REQUIRED FINAL STEPS

Important: this is a documented draft, not a verified 15/15 submission

- Run all notebooks and save charts, SQL output tables, Folium screenshots and model results.
- Add missing Plotly Dash application / screenshots and actual success pie + payload scatter.
- Replace result-status statements with verified numbers and interpretations.
- Upload all completed notebooks and Python scripts to a public GitHub repository; add its real URL to slides.
- Export this edited presentation as PDF; upload PDF and provide GitHub URL in the course submission.

No GitHub URL was supplied; none has been invented.